

Basics

Yes. The important thing is to **build a map of the OOP entities first**, rather than jumping straight into inheritance/polymorphism.

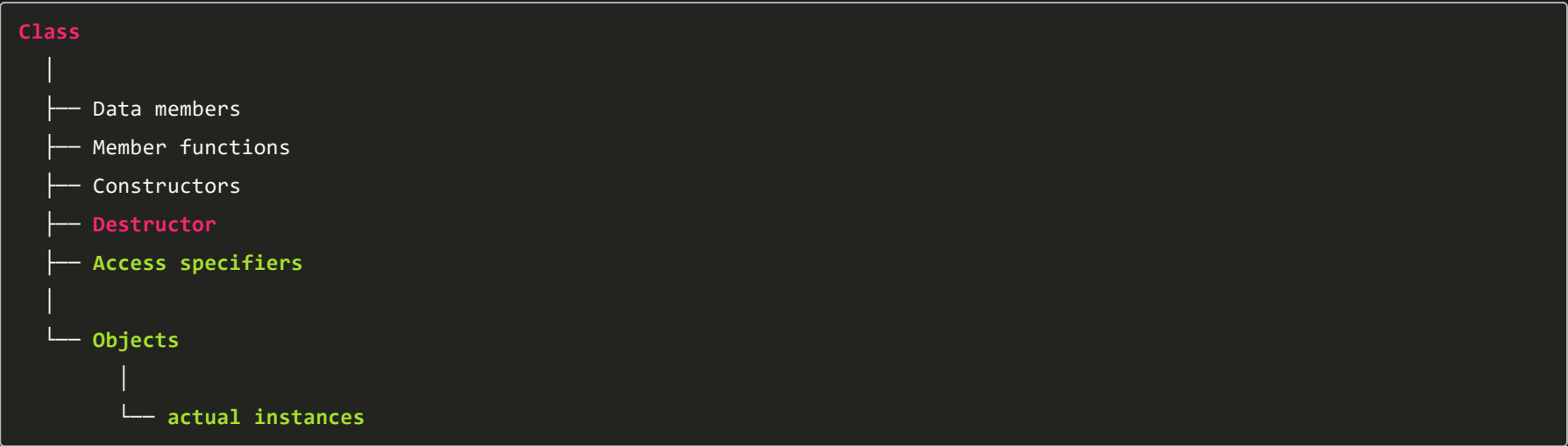
One terminology correction first:

- **Class** → yes
- **Abstract class** → yes
- **Interface** → C++ doesn't have a dedicated `interface` keyword; we create interfaces using **pure virtual functions**
- **Virtual class** → this is usually a misunderstanding. There is no general "virtual class" in C++. You probably mean **virtual functions** or **abstract classes**
- **Object** → instance of a class
- **Struct** → class-like type with different default access
- **Enum** → user-defined set of named constants
- **Inheritance** → relationship between classes
- **Composition/Aggregation** → objects containing/using other objects
- **Templates** → generic classes/functions

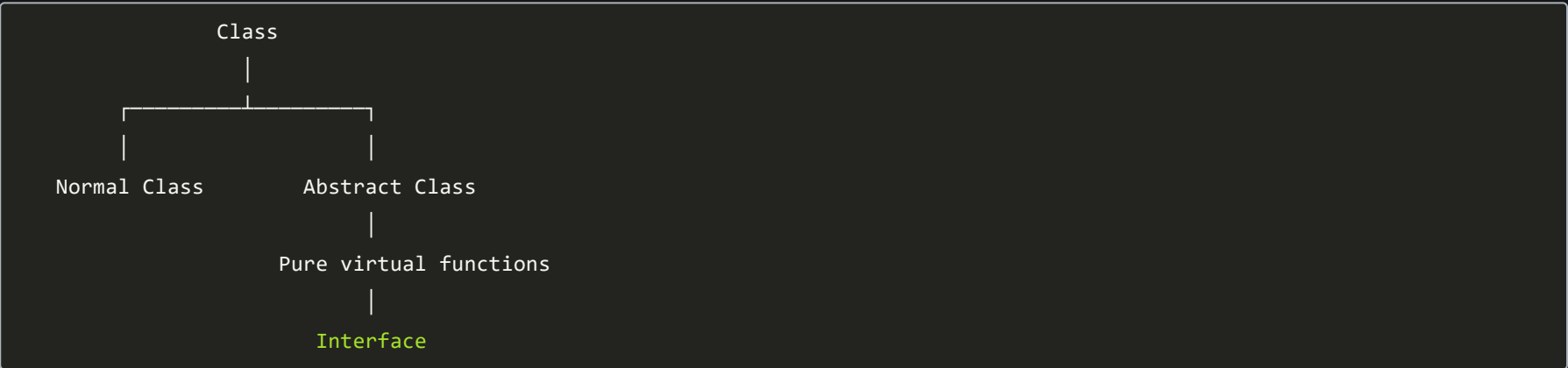
Let's build these from the ground up.

1. The basic OOP picture

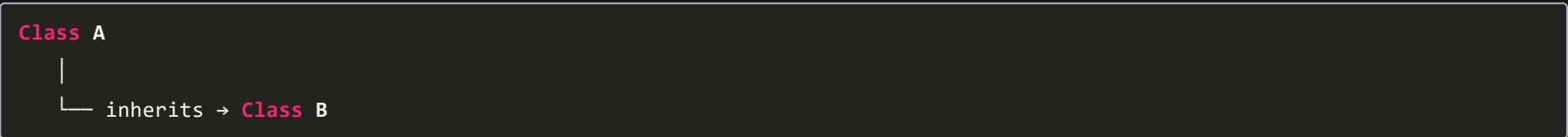
Think of OOP in C++ as:



Then classes can relate to each other:



And:



Let's learn each entity.

2. Class

A **class is a blueprint** for creating objects.

```
class Student {
public:
    string name;
    int age;

    void study() {
        cout << name << " is studying";
    }
};
```

Now create an object:

```
Student s1;

s1.name = "Lalit";
s1.age = 23;

s1.study();
```

Think:

```
Student      → blueprint
s1           → object
s1.name      → data member
s1.study()   → member function
```

3. Object

An object is an **instance of a class**.

```
class Car {
public:
    string brand;

    void drive() {
        cout << "Car is driving";
    }
};
```

Create objects:

```
Car c1;
Car c2;
```

Now:

```
Car
|
├─ c1
└─ c2
```

Both objects have their own **brand**.

```
c1.brand = "BMW";
c2.brand = "Audi";
```

But both use the same class definition.

4. Access Specifiers

Inside a class, you'll constantly encounter:

```
public
private
protected
```

public

Accessible from outside the class.

```
class Student {
public:
    string name;
};

Student s;
s.name = "Lalit";
```

private

Accessible only inside the class.

```
class Student {
private:
    int age;
};
```

This won't work:

```
Student s;  
s.age = 20;    // ERROR
```

protected

Accessible inside the class **and derived classes**.

```
class Parent {  
protected:  
    int x;  
};
```

We'll understand why **protected** matters when we learn inheritance.

5. Encapsulation

Encapsulation means **bundling data + functions together and controlling access to the data**.

For example:

```
class BankAccount {  
private:  
    double balance;  
  
public:  
    void deposit(double amount) {  
        balance += amount;  
    }  
  
    double getBalance() {  
        return balance;  
    }  
};
```

You don't allow this:

```
account.balance = -100000; // protected from direct access
```

Instead:

```
account.deposit(5000);  
cout << account.getBalance();
```

This is one of the fundamental ideas of OOP.

6. Constructor

A constructor is a special function that runs automatically when an object is created.

```
class Student {  
public:  
    string name;  
    int age;  
  
    Student(string n, int a) {  
        name = n;  
        age = a;  
    }  
};
```

Now:

```
Student s("Lalit", 23);
```

The constructor automatically executes.

Better way: initializer list

You'll commonly see:

```
class Student {
private:
    string name;
    int age;

public:
    Student(string name, int age)
        : name(name), age(age) {}
};
```

This is the preferred C++ style.

7. Destructor

Destructor runs when an object is destroyed.

```
class Student {
public:
    ~Student() {
        cout << "Object destroyed";
    }
};
```

Syntax:

```
~ClassName()
```

No return type, no parameters.

You'll see destructors become **very important with inheritance and polymorphism**.

8. `this` pointer

Inside a member function, `this` points to the current object.

```
class Student {
private:
    string name;

public:
    Student(string name) {
        this->name = name;
    }
};
```

Here:

```
name        → parameter
this->name   → object's member
```

9. Static members

Normally, every object gets its own data.

```
class Student {
public:
    int age;
};
s1.age
s2.age
s3.age
```

But `static` means **shared by the class**.

```
class Student {
public:
    static int count;
};

int Student::count = 0;
```

Now:

```
Student::count
```

is shared among all objects.

10. Struct

C++ `struct` is very similar to a class.

```
struct Student {
    string name;
    int age;
};
```

The major default difference:

```
class
→ members private by default

struct
→ members public by default
```

For example:

```
class A {
    int x;           // private
};

struct B {
    int x;           // public
};
```

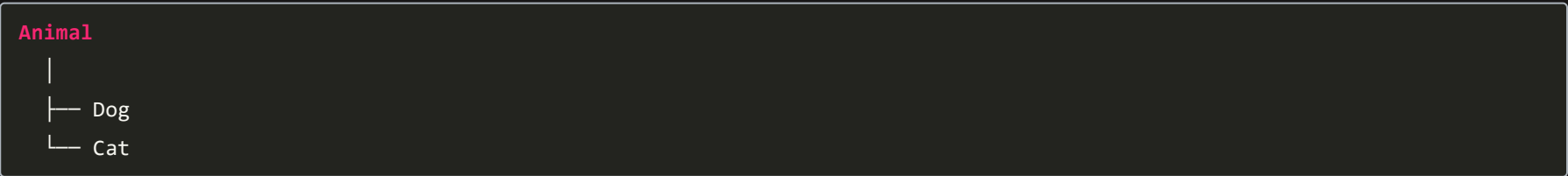
Both can have:

- constructors
- functions
- inheritance
- private/protected/public
- static members
- etc.

11. Inheritance

Now we start getting into the really important OOP stuff.

Suppose:



In C++:

```
class Animal {
public:
    void eat() {
        cout << "Eating";
    }
};

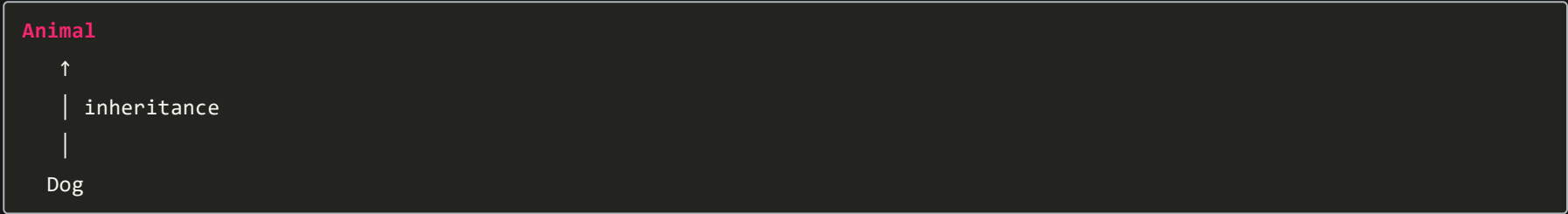
class Dog : public Animal {
public:
    void bark() {
        cout << "Barking";
    }
};
```

Now:

```
Dog d;

d.eat();    // inherited
d.bark();   // own function
```

So:



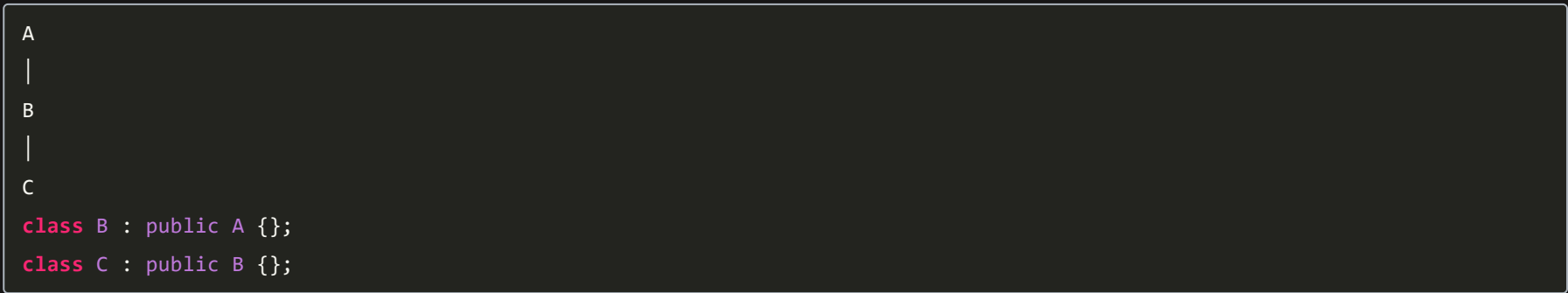
12. Types of inheritance

You'll encounter:

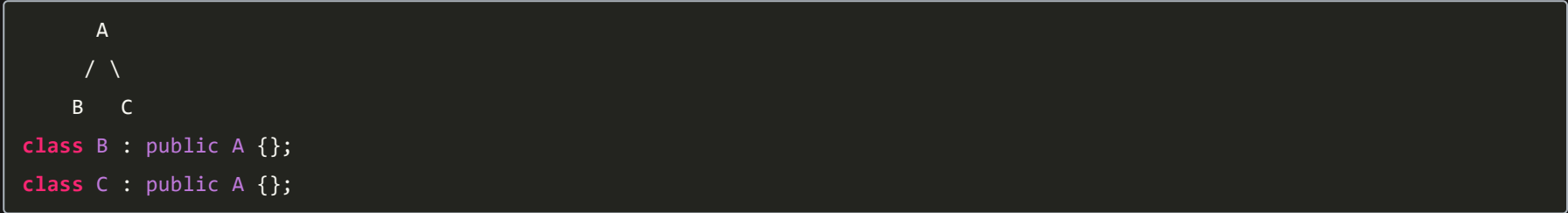
Single



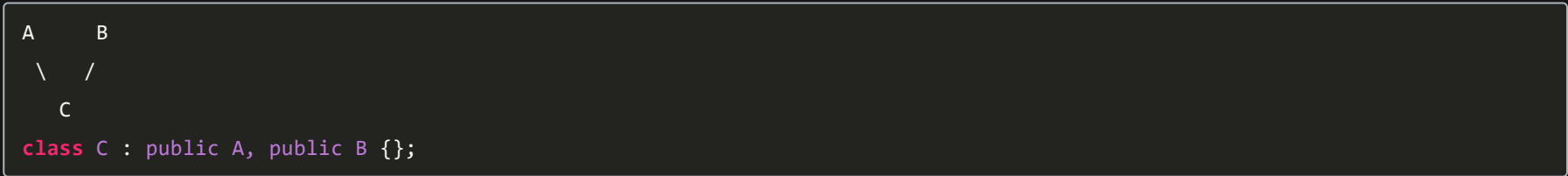
Multilevel



Hierarchical



Multiple



Hybrid

Combination of multiple inheritance types.

13. Method overriding

Suppose:

```
class Animal {
public:
    void sound() {
        cout << "Animal sound";
    }
};

class Dog : public Animal {
public:
    void sound() {
        cout << "Bark";
    }
};
```

Dog provides its own version of **sound()**.

This is **method overriding**.

But there's an important problem.

```
Animal* a = new Dog();

a->sound();
```

Without `virtual`, C++ will call:

```
Animal::sound()
```

rather than:

```
Dog::sound()
```

That's where **virtual functions** come in.

14. Virtual function

```
class Animal {
public:
    virtual void sound() {
        cout << "Animal sound";
    }
};
```

Now:

```
class Dog : public Animal {
public:
    void sound() override {
        cout << "Bark";
    }
};
```

Then:

```
Animal* a = new Dog();

a->sound();
```

Output:

```
Bark
```

This is **runtime polymorphism**.

The key mental model:

```
Animal* a
  |
  | points to
  ↓
Dog object

virtual
  ↓
choose function based on ACTUAL object
```

15. `override`

Whenever you override a virtual function, preferably write:

```
void sound() override
```

instead of:

```
void sound()
```

Why?

Because the compiler verifies that you're actually overriding a base-class virtual function.

16. Pure virtual function

Now we get to **abstract classes** and **interfaces**.

Suppose every animal must have a `sound()` but we don't want to provide an implementation in `Animal`.

Write:

```
class Animal {
public:
    virtual void sound() = 0;
};
```

This is a **pure virtual function**.

The:

```
= 0
```

is the important part.

17. Abstract class

A class containing at least one pure virtual function is an **abstract class**.

```
class Animal {
public:
    virtual void sound() = 0;
};
```

You cannot do:

```
Animal a;    // ERROR
```

Because `Animal` is abstract.

But you can:

```
class Dog : public Animal {
public:
    void sound() override {
        cout << "Bark";
    }
};
```

Then:

```
Dog d;
d.sound();
```

18. Interface in C++

C++ does **not** have:

```
interface Animal
```

like Java/C#.

Instead, we usually create an interface using a class containing pure virtual functions.

Example:

```
class Animal {
public:
    virtual void sound() = 0;
    virtual void eat() = 0;

    virtual ~Animal() = default;
};
```

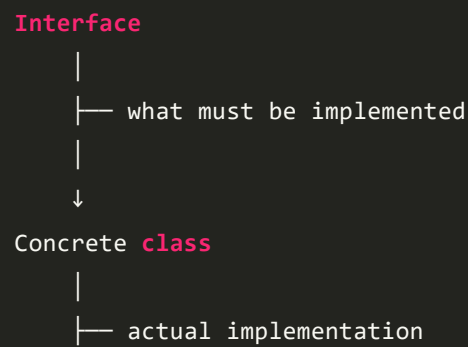
Then:

```
class Dog : public Animal {
public:
    void sound() override {
        cout << "Bark";
    }

    void eat() override {
        cout << "Dog eating";
    }
};
```


This behaves like an interface.

Mental model:



19. Abstract class vs Interface

This is an important interview distinction.

Abstract class

Can contain:

```
normal functions
pure virtual functions
data members
constructors
destructor
```

Example:

```
class Animal {
protected:
    int age;

public:
    void sleep() {
        cout << "Sleeping";
    }

    virtual void sound() = 0;
};
```

Interface-style class

Usually contains mostly/all pure virtual functions:

```
class Printable {
public:
    virtual void print() = 0;
    virtual ~Printable() = default;
};
```

So:

```
Abstract class
= partially implemented base class

Interface
= contract specifying what derived classes must implement
```

C++ doesn't enforce the **interface** keyword distinction.

20. Multiple interfaces

This is one place where C++'s multiple inheritance becomes useful.

```
class Printable {
public:
    virtual void print() = 0;
};

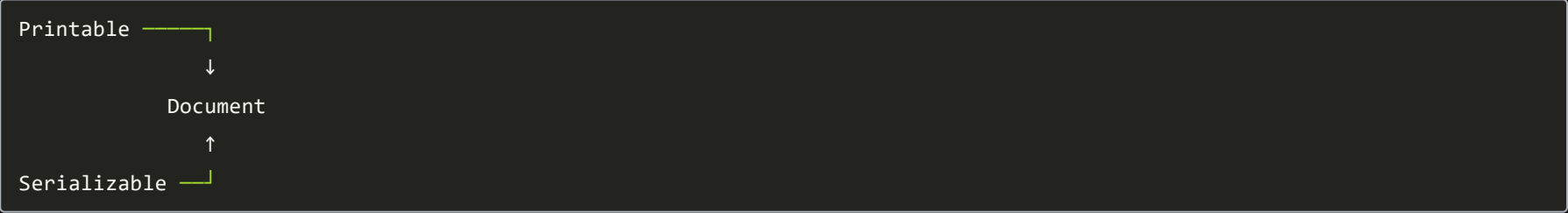
class Serializable {
public:
    virtual void serialize() = 0;
};
```

A class can implement both:

```
class Document : public Printable, public Serializable {
public:
    void print() override {
        cout << "Printing";
    }

    void serialize() override {
        cout << "Serializing";
    }
};
```

So:



21. Virtual destructor

Very important interview concept.

Consider:

```
class Animal {
public:
    virtual ~Animal() = default;
};
```

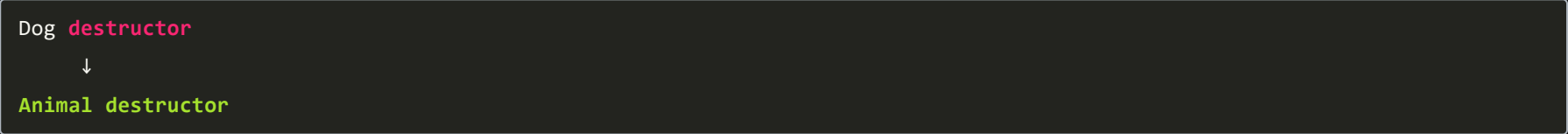
Why virtual?

Because:

```
Animal* a = new Dog();

delete a;
```

You want:



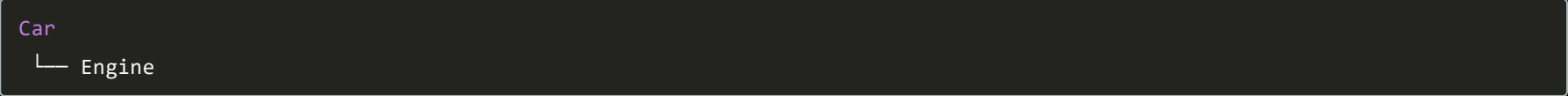
to happen correctly.

Therefore, if a class is intended to be used polymorphically, its destructor should generally be virtual.

22. Composition

Not every relationship should be inheritance.

Suppose:



A car **has an** engine.

That's composition.

```

class Engine {
public:
    void start() {
        cout << "Engine started";
    }
};

class Car {
private:
    Engine engine;

public:
    void startCar() {
        engine.start();
    }
};

```

Mental rule:

```

IS-A → inheritance

HAS-A → composition
Dog IS-A Animal
Car HAS-A Engine

```

This distinction is extremely important in OOP design.

23. Aggregation

Aggregation is also a **HAS-A relationship**, but the contained object can exist independently.

For example:

```

Department
|
└─ Professors

```

A professor can exist without the department.

Conceptually:

```

class Professor {};

class Department {
private:
    Professor* professor;
};

```

The distinction between composition and aggregation is mainly about **ownership/lifetime**.

```

Composition
→ strong ownership

Aggregation
→ weaker relationship

```

24. Association

Association is the most general relationship.

Two objects simply know/use each other.

```

Student — Teacher

```

For example:

```

class Teacher {};

class Student {
public:
    void learnFrom(Teacher& teacher) {
        // use teacher
    }
};

```

No ownership is necessarily involved.

25. The complete OOP entity map

This is the map I'd recommend memorizing:



26. The 4 pillars fit into this

Now you can see where the famous **4 pillars of OOP** fit.

Encapsulation

```
class BankAccount {
private:
    double balance;

public:
    void deposit(double amount);
};
```

Bundle + protect data.

Abstraction

```
class Animal {
public:
    virtual void sound() = 0;
};
```

Expose what is necessary, hide implementation details.

Inheritance

```
class Dog : public Animal {};
```

Reuse/extend existing class.

Polymorphism

```
Animal* a = new Dog();
a->sound();
```

Same interface, different behavior.


```
class
object
constructor
destructor
protected
public
inheritance
abstract class
pure virtual function
virtual function
method overriding
override
runtime polymorphism
base pointer
derived object
```

28. What you should learn next

Don't try to memorize all of this at once. For your C++ OOP interview preparation, I'd go in this exact order:

```
1. Class & Object
    ↓
2. Data members & Member functions
    ↓
3. public / private / protected
    ↓
4. Constructor
    ↓
5. Destructor
    ↓
6. this pointer
    ↓
7. static members
    ↓
8. Encapsulation
    ↓
9. Inheritance
    ↓
10. Types of inheritance
    ↓
11. Function overriding
    ↓
12. Virtual functions
    ↓
13. Pure virtual functions
    ↓
14. Abstract classes
    ↓
15. Interfaces in C++
    ↓
16. Virtual destructor
    ↓
17. Polymorphism
    ↓
18. Composition / Aggregation / Association
    ↓
19. Multiple inheritance + Diamond problem
    ↓
20. Virtual inheritance
```

After that, move into the more C++-specific OOP topics:


```
Rule of 3
Rule of 5
Rule of 0
Copy constructor
Copy assignment
Move constructor
Move assignment
Operator overloading
Friend functions/classes
Templates
Exception handling
RAII
Smart pointers
```

That gives you a much more complete picture of "what entities/things exist in C++ OOP and how they are written", rather than just memorizing the four pillars.